Apache Kafka – Engineering Notes

1. Introduction to Apache Kafka

Apache Kafka is a distributed event streaming platform used to build real-time data pipelines and streaming applications.

Kafka allows applications to publish, store, and consume streams of records efficiently.

It is designed to handle high-throughput data with low latency and fault tolerance.

2. Kafka Topics

A topic is a logical category or feed name to which records are published.

For example, an e-commerce application might have topics such as:

- orders
- payments
- customer-events
- inventory-updates

Producers publish messages to topics, and consumers subscribe to topics to receive those messages.

3. Kafka Partitions

A Kafka topic is divided into one or more partitions.

Partitions allow Kafka to distribute data across multiple brokers and enable parallel message processing.

Each message within a partition receives an offset, which identifies its position in that partition.

4. Kafka Replication

Kafka supports replication to provide fault tolerance and high availability.

A partition can have multiple replicas distributed across different brokers.

One replica acts as the leader, while the others act as followers.

Producers and consumers normally communicate with the leader replica.

If the leader broker fails, Kafka can elect another replica as the new leader.

5. Kafka Producers and Consumers

A producer is an application that publishes messages to Kafka topics.

A consumer is an application that reads messages from Kafka topics.

Producers and consumers are independent of each other, which helps build loosely coupled distributed systems.

6. Consumer Groups

A consumer group consists of multiple consumers working together to consume messages from one or more topics.

Kafka distributes partitions among consumers in the same group.

This allows multiple consumers to process data in parallel and improves scalability.

7. Real-World Example

Consider an e-commerce application.

When a customer places an order:

1. The order service publishes an order event to the Kafka orders topic.
2. The payment service consumes the event and processes the payment.
3. The inventory service consumes the event and updates stock.
4. The notification service consumes the event and sends an order confirmation.

Kafka enables these services to communicate asynchronously without directly depending on each other.

8. Key Takeaways

- Kafka is a distributed event streaming platform.
- Topics organize messages into logical categories.
- Partitions enable parallel processing and scalability.
- Replication provides fault tolerance.
- Producers publish messages.

- Consumers read messages.
- Consumer groups allow multiple consumers to share processing work.